Laboratório 05:

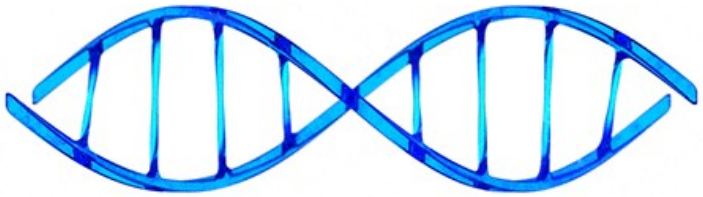
Algoritmo Genético

CC7711 - Inteligência Artificial

Prof. Dr. Murillo | Depto. de Ciência da Computação

Definição de um indivíduo (Código Genético)

- **Cromossomos:** Uma solução candidata (representa um ponto no espaço de busca).
- **População:** O conjunto de indivíduos que evoluem a cada iteração.
- **Função de Aptidão (Fitness):** Função de custo que mede o quão próximo a solução está do objetivo ideal.



Processo Evolutivo

- **Seleção:** Escolhe os cromossomos aleatoriamente priorizando os com melhor fitness (método da roleta, por exemplo).
- **Crossover:** Combina as soluções entre os melhores cromossomos para explorar regiões próximas da solução ótima.
- **Mutação:** Altera os genes aleatoriamente para garantir variabilidade na exploração do espaço de busca.

> **Objetivo Central:** Selecionar o cromossomo com o maior valor de fitness para chegar à solução ótima global.

Classe Algoritmo Genético

```
class AlgoritmoGenetico:
    def __init__(self, tamanho_populacao, taxa_mutacao, tamanho_elitismo=2):
        self.tamanho_populacao = tamanho_populacao
        self.taxa_mutacao = taxa_mutacao
        self.tamanho_elitismo = tamanho_elitismo
        self.populacao = []

    # Métodos que DEVEM ser sobrescritos pelas classes filhas
    def gerar_individuo(self):
        raise NotImplementedError("Os alunos precisam definir como é o cromossomo!")

    def calcular_fitness(self, individuo):
        raise NotImplementedError("Os alunos precisam definir a função de aptidão!")

    def crossover(self, pai1, pai2):
        raise NotImplementedError("Os alunos precisam definir como os pais se reproduzem!")

    def mutacao(self, individuo):
        raise NotImplementedError("Os alunos precisam definir a mutação para gerar diversidade!")
```

Classe Algoritmo Genético

```
def inicializar_populacao(self):
    self.populacao = [self.gerar_individuo() for _ in range(self.tamanho_populacao)]

def selecao_roleta(self, fitness_populacao):
    soma_fitness = sum(fitness_populacao)
    if soma_fitness == 0:
        return random.choice(self.populacao)
    probabilidades = [f / soma_fitness for f in fitness_populacao]
    indice_escolhido = np.random.choice(len(self.populacao), p=probabilidades)
    return self.populacao[indice_escolhido]

def evoluir(self, max_geracoes, objetivo_fitness=None, paciencia=None):
    self.inicializar_populacao()
            #Completar o método evoluir
```

Desafio 1: Equação Algébrica

Utilizar o Algoritmo Genético para encontrar os valores de X, Y e Z para a seguinte equação:

$$x + y + z = 3$$

Desenvolver com Herança: Criar a classe EquacaoLinear que herda da classe AlgoritmoGenetico.

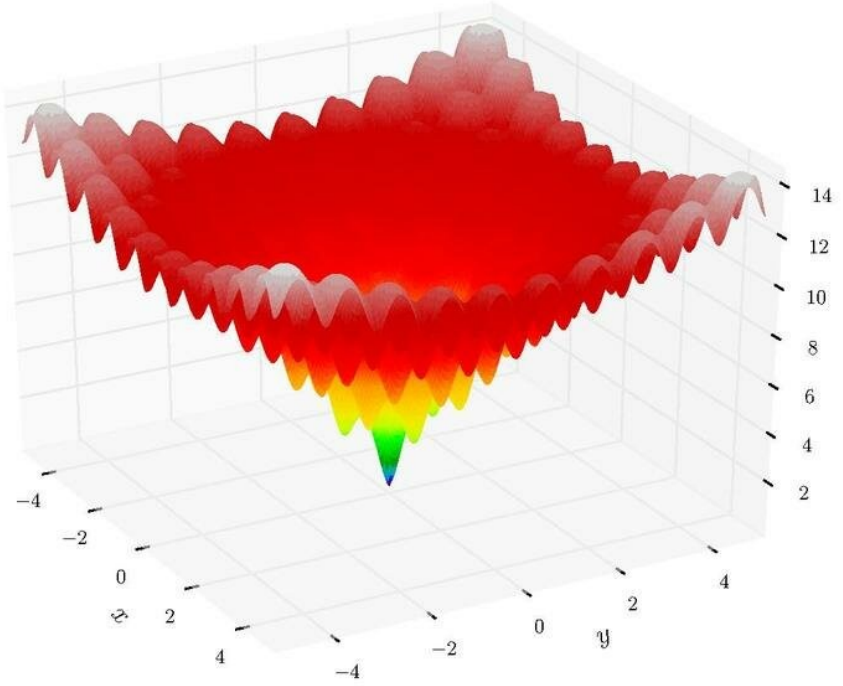
```
class EquacaoLinear(AlgoritmoGenetico):
    def __init__(self, tamanho_populacao, taxa_mutacao):
        super().__init__(tamanho_populacao, taxa_mutacao)
        self.dimensoes = 3 #variáveis: x, y, z

    def gerar_individuo(self):
        # Gera valores aleatórios iniciais entre -10 e 10
        return np.random.uniform(-10, 10, self.dimensoes)

    def calcular_fitness(self, individuo):
        x, y, z = individuo
        erro = abs((x + y + z) - 3.0)
        return 1.0 / (erro + 1e-6)
```

Desafio 2: Função de Ackley

Utilizar o Algoritmo Genético para otimizar a função de Ackley em duas dimensões (x, y).

Função clássica para benchmark de algoritmos de otimização por conta de mínimos locais espalhados pelo espaço de busca

$$f(x, y) = -20\exp\left(-0.2\sqrt{0.5(x^2 + y^2)}\right) - \exp\left(0.5(\cos(2\pi x) + \cos(2\pi y))\right) + e + 20$$



Classe Ackley: Resolução similar à EquacaoLinear, apenas alterar a equação.

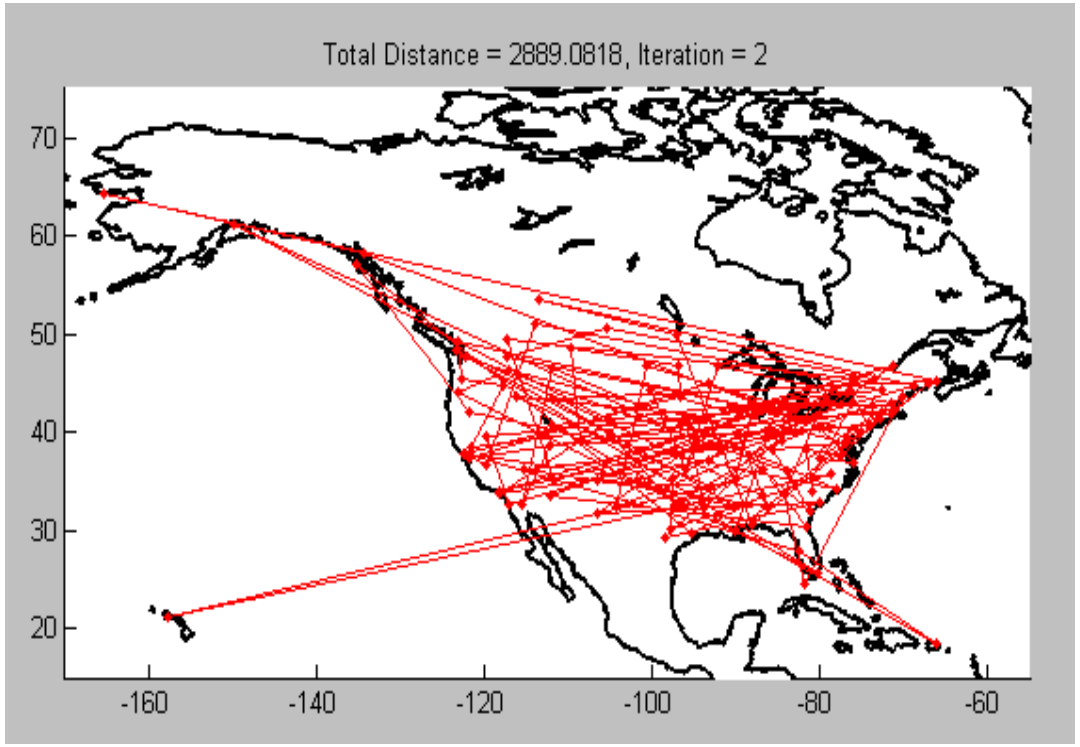
```
class Ackley(AlgoritmoGenetico):
    def __init__(self, tamanho_populacao, taxa_mutacao):
        super().__init__(tamanho_populacao, taxa_mutacao)
        self.dimensoes = 2
    def gerar_individuo(self):
        return np.random.uniform(-32.0, 32.0, self.dimensoes)
```

Desafio 3: Problema do caixeiro viajante

Utilizar o Algoritmo Genético para resolver o problema do caixeiro viajante.

Dado um conjunto de coordenadas (x, y), que representa a posição das cidades, calcular o caminho de menor distância que passe por todas as cidades uma única vez.

```
class CaixeiroViajante(AlgoritmoGenetico):
    def __init__(self, coordenadas_cidades, tamanho_populacao, taxa_mutacao):
        super().__init__(tamanho_populacao, taxa_mutacao)
        self.cidades = coordenadas_cidades
        self.num_cidades = len(coordenadas_cidades)
```

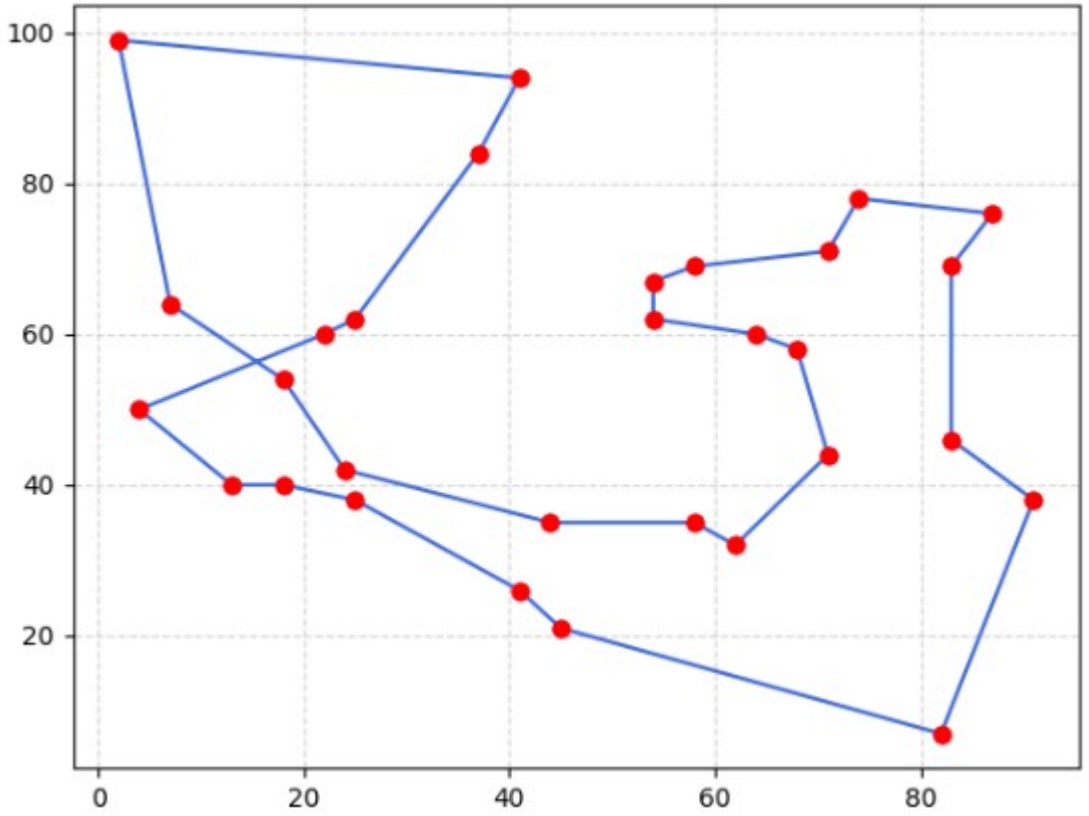


Desafio 3: Problema do caixeiro viajante

Caso de teste:

```
cidades_tsp = np.array([
    [2.0, 99.0], [4.0, 50.0], [7.0, 64.0], [13.0,
40.0], [18.0, 54.0],
    [18.0, 40.0], [22.0, 60.0], [24.0, 42.0], [25.0,
62.0], [25.0, 38.0],
    [37.0, 84.0], [41.0, 94.0], [41.0, 26.0], [44.0,
35.0], [45.0, 21.0],
    [54.0, 67.0], [54.0, 62.0], [58.0, 69.0], [58.0,
35.0], [62.0, 32.0],
    [64.0, 60.0], [68.0, 58.0], [71.0, 71.0], [71.0,
44.0], [74.0, 78.0],
    [82.0, 7.0],  [83.0, 69.0], [83.0, 46.0], [87.0,
76.0], [91.0, 38.0]
])
```

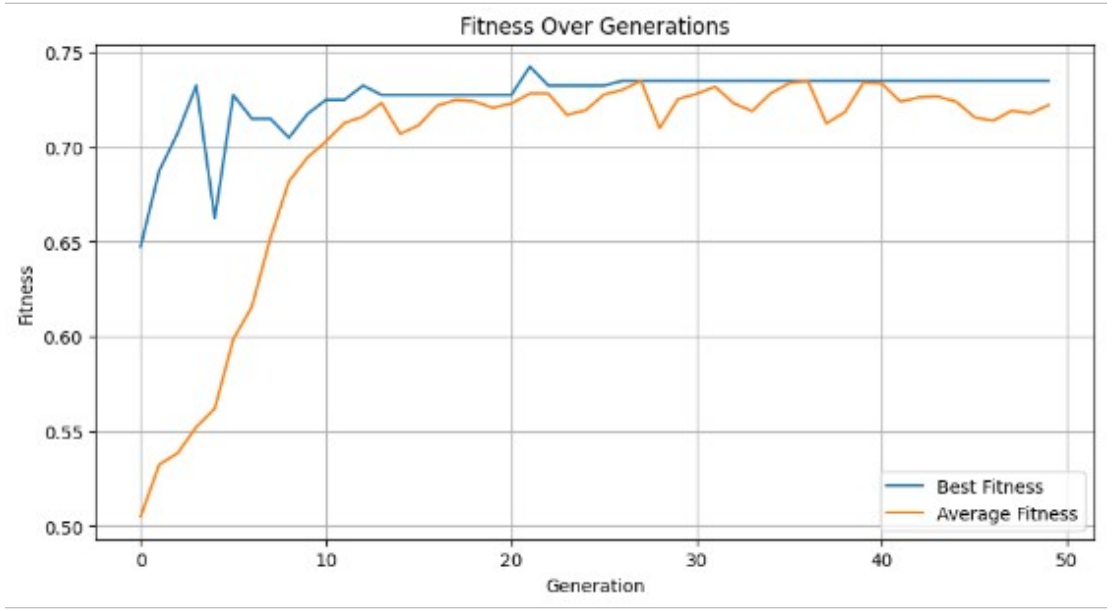
Solução Obtida:



Métricas e Convergência

Monitorar o valor de fitness para cada iteração da evolução dos cromossomos.

- **Gráfico de Convergência:** Evolução do Melhor Fitness vs Gerações.
- **Early Stopping:** Encerra o processo evolutivo se o valor de fitness não tem mais melhorias.



Roteiro do Laboratório & Entregas

Utilizem o template de código Python (.ipynb) fornecido para completar as seguintes tarefas:

- **Passo 1:** Resolver a Equação Algébrica utilizando a classe EquacaoLinear que herda da classe AlgoritmoGenetico.
- **Passo 2:** Resolver a Função de Ackley utilizando a classe Ackley (plotar gráfico de convergência)
- **Passo 3:** Resolver o problema do Caixeiro Viajante utilizando a classe Caixeiro Viajante (plotar gráfico de convergência e solução gráfica)

Métricas e Submissão: Submetam o notebook com o código completo. Ao final, apresentem a solução dos 3 problemas com as métricas: **Tempo Total (s)**, **Função Fitness**, **Gráfico de Convergência** e **Solução Gráfica (para o Caixeiro Viajante)**.